Project: Vehicle Insurance Policy Administration System

1. Overview

The **Vehicle Insurance Policy Administration System** is designed to manage vehicle insurance operations such as policy issuance, premium calculation, accident claim handling, renewal tracking, and customer support. The system ensures that vehicle owners and administrators can efficiently manage all aspects of vehicle insurance while adhering to industry standards. This application leverages the **MVC architecture**, making it compatible with both **Java (Spring MVC)** and **.NET (ASP.NET Core MVC)** frameworks.

The system comprises the following modules:

1. **Customer Management**
2. **Policy Management**
3. **Premium Calculator**
4. **Accident Claim Management**
5. **Renewal Management**

2. Assumptions

- Role-based access for customers, agents, and administrators.
- Policy-related data will include vehicle information, insurance type, and premium details.
- SQL Server or MySQL will be used for database storage.
- The application interface will be web-based for ease of access.

3. Module-Level Design

3.1 Customer Management Module

Purpose: Handles customer information and user registration.

- **Controller:**
  - CustomerController
    - registerCustomer()
    - updateCustomerDetails()
    - getCustomerDetails()
    - getAllCustomers()
- **Service:**
  - CustomerService
    - Handles business logic related to customer data management.
- **Model:**
  - **Customer Entity**

        - Attributes:
            - customerId (PK)
            - name
            - email
            - contactNumber
            - address
            - userRole (ADMIN, AGENT, CUSTOMER)

3.2 Policy Management Module

Purpose: Manages vehicle insurance policies, including their creation, retrieval, and updates.

- **Controller:**
    - PolicyController
        - createPolicy()
        - getPolicyDetails()
        - updatePolicy()
        - deletePolicy()
        - getAllPolicies()
- **Service:**
    - PolicyService
        - Implements business rules for policy issuance and updates.
- **Model:**
    - **Policy Entity**
        - Attributes:
            - policyId (PK)
            - customerId (FK)
            - vehicleId (FK)
            - policyNumber
            - policyStartDate
            - policyEndDate
            - premiumAmount
            - policyStatus (ACTIVE, EXPIRED)

3.3 Premium Calculator Module

Purpose: Calculates premiums based on vehicle type, coverage, and risk factors.

- **Controller:**
    - PremiumCalculatorController
        - calculatePremium()
- **Service:**
    - PremiumCalculatorService
        - Contains algorithms for premium computation based on vehicle age, type, and owner's risk profile.
- **Model:**
    - No entity required; operates on policy and customer data.

3.4 Accident Claim Management Module

Purpose: Manages the submission, processing, and status tracking of accident claims.

- **Controller:**
  - ClaimController
    - submitClaim()
    - getClaimDetails()
    - updateClaimStatus()
    - getAllClaims()
- **Service:**
  - ClaimService
    - Implements claim validation, approval, or rejection processes.
- **Model:**
  - **Claim Entity**
    - Attributes:
      - claimId (PK)
      - policyId (FK)
      - claimAmount
      - claimReason
      - claimDate
      - claimStatus (SUBMITTED, APPROVED, REJECTED)

3.5 Renewal Management Module

Purpose: Tracks policy renewal deadlines and facilitates renewal processes.

- **Controller:**
  - RenewalController
    - initiateRenewal()
    - getRenewalDetails()
    - updateRenewalStatus()
- **Service:**
  - RenewalService
    - Handles renewal notifications and processes renewal updates.
- **Model:**
  - **Renewal Entity**
    - Attributes:
      - renewalId (PK)
      - policyId (FK)
      - renewalDate
      - newPremiumAmount
      - renewalStatus (PENDING, COMPLETED)

4. Database Schema

4.1 Table Definitions

1. **Customer Table**

   CREATE TABLE Customer (

```
    customerId INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    contactNumber VARCHAR(15),
    address TEXT,
    userRole ENUM('ADMIN', 'AGENT', 'CUSTOMER')
);
```

2. **Vehicle Table**

```
CREATE TABLE Vehicle (
    vehicleId INT AUTO_INCREMENT PRIMARY KEY,
    customerId INT,
    registrationNumber VARCHAR(20),
    vehicleType ENUM('CAR', 'BIKE', 'TRUCK'),
    vehicleAge INT,
    FOREIGN KEY (customerId) REFERENCES Customer(customerId)
);
```

3. **Policy Table**

```
CREATE TABLE Policy (
    policyId INT AUTO_INCREMENT PRIMARY KEY,
    customerId INT,
    vehicleId INT,
    policyNumber VARCHAR(50),
    policyStartDate DATE,
    policyEndDate DATE,
    premiumAmount DECIMAL(10, 2),
    policyStatus ENUM('ACTIVE', 'EXPIRED'),
    FOREIGN KEY (customerId) REFERENCES Customer(customerId),
    FOREIGN KEY (vehicleId) REFERENCES Vehicle(vehicleId)
);
```

4. **Claim Table**

```
CREATE TABLE Claim (
    claimId INT AUTO_INCREMENT PRIMARY KEY,
    policyId INT,
    claimAmount DECIMAL(10, 2),
    claimReason TEXT,
    claimDate DATE,
    claimStatus ENUM('SUBMITTED', 'APPROVED', 'REJECTED'),
    FOREIGN KEY (policyId) REFERENCES Policy(policyId)
);
```

5. **Renewal Table**

```
CREATE TABLE Renewal (
```

```
    renewalId INT AUTO_INCREMENT PRIMARY KEY,
    policyId INT,
    renewalDate DATE,
    newPremiumAmount DECIMAL(10, 2),
    renewalStatus ENUM('PENDING', 'COMPLETED'),
    FOREIGN KEY (policyId) REFERENCES Policy(policyId)
);
```

5. Local Deployment Details

1. **Environment Setup:**
    - Install MySQL or SQL Server and configure the database schema using the provided scripts.
    - Set up the application runtime environment: JDK (Java) or .NET SDK.
    - Configure application settings for database connections.
2. **Deployment Steps:**
    - Clone the project repository.
    - Build the project using Maven (Java) or Visual Studio (ASP.NET Core).
    - Deploy the application on Tomcat (Java) or IIS/Kestrel (ASP.NET Core).
    - Run the application on localhost and test using browser or Postman.

6. Conclusion

The **Vehicle Insurance Policy Administration System** is a scalable, modular solution tailored for managing vehicle insurance operations. Its adherence to the **MVC architecture** ensures compatibility with both **Java** and **.NET** ecosystems. The modularity supports seamless maintenance and scalability, with comprehensive database designs supporting efficient data storage and retrieval.